

Documentazione Test

1. Test di Unità

Package: model

Classe Testata: NodeDTO

- **Test 1 (testInternalNodeCreation):**
 - **Cosa testa:** L'inizializzazione corretta dello stato per un nodo interno di split.
 - **Descrizione:** Istanza un NodeDTO tramite il costruttore a cui viene passata una condizione di split (es. "X <= 10"). Il test verifica che il campo leaf sia impostato a false, che splitCondition contenga la stringa attesa, che predictedValue sia null e che la collezione dei nodi figli sia inizializzata come lista vuota.
- **Test 2 (testLeafNodeCreation):**
 - **Cosa testa:** L'inizializzazione dello stato per un nodo foglia.
 - **Descrizione:** Istanza un NodeDTO tramite il costruttore numerico con un valore stimato (es. 42.5). Il test verifica che il campo leaf risulti true, che predictedValue coincida con il valore passato e che splitCondition sia null.
- **Test 3 (testAddChild):**
 - **Cosa testa:** L'inserimento di un nodo figlio nella struttura dell'albero.
 - **Descrizione:** Crea un nodo interno e vi aggiunge un nodo secondario tramite addChild(). Il test verifica che la lista restituita da getChildren() aumenti la propria dimensione a 1 e contenga esattamente il riferimento al nodo inserito.
- **Test 4 (testMultipleChildren):**
 - **Cosa testa:** La conservazione dell'ordine di inserimento su inserimenti multipli di nodi figli.
 - **Descrizione:** Inserisce sequenzialmente più nodi figli all'interno dello stesso nodo padre. Il test verifica che la lista children mantenga inalterato l'ordine di aggiunta dei rami decisionali.
- **Test 5 (testDefaultSelected):**
 - **Cosa testa:** Lo stato predefinito del flag di selezione visiva.
 - **Descrizione:** Istanza un nuovo nodo e verifica che la proprietà booleana selected sia valorizzata a true di default, garantendo la visibilità del nodo durante il rendering grafico.
- **Test 6 (testToStringInternalNode):**
 - **Cosa testa:** La rappresentazione testuale del nodo interno.
 - **Descrizione:** Invoca il metodo toString() su un nodo non foglia. Il test verifica che la stringa restituita coincida esattamente con la splitCondition memorizzata.
- **Test 7 (testToStringLeafNode):**
 - **Cosa testa:** La formattazione testuale del risultato predittivo su un nodo foglia.
 - **Descrizione:** Invoca toString() su un nodo foglia. Il test accerta che la stringa contenga il prefisso formattato "Predizione: <valore>" con il corrispondente valore di stima.
- **Test 8 (testAddChildToLeafNode):**
 - **Cosa testa:** La gestione dell'incoerenza strutturale in caso di aggiunta di rami a una foglia.
 - **Descrizione:** Tenta di invocare addChild() su un nodo istanziato come foglia. Il test verifica che l'operazione modifichi coerentemente lo stato leaf a false (o sollevi IllegalStateException in base ai vincoli implementativi), impedendo nodi foglia con figli attivi.

- **Test 9 (testToStringWithNullValues):**

- **Cosa testa:** La robustezza di toString() a fronte di campi non valorizzati.
- **Descrizione:** Invoca toString() su nodi aventi splitCondition o predictedValue impostati a null. Il test verifica che il metodo gestisca l'assenza di dati senza sollevare NullPointerException.

Classe Testata: TreeDTO

- **Test 1 (testConstructor):**

- **Cosa testa:** L'aggregazione iniziale della radice e del nome tabella associato.
- **Descrizione:** Istanza TreeDTO passando un NodeDTO e il nome di una tabella di training. Il test verifica che i metodi getRoot() e getTableName() restituiscano fedelmente i valori forniti al costruttore.

Package: view

Classe Testata: ControlPanel

- **Test 1 (testDefaultValues):**

- **Cosa testa:** La corretta impostazione dei controlli grafici all'avvio.
- **Descrizione:** Verifica che all'istanziamento del pannello l'indirizzo IP predefinito sia "127.0.0.1", la porta sia 8080, la tabella selezionata di default sia "provac" e il pulsante per la predizione risulti disabilitato.

- **Test 2 (testGetServerPort):**

- **Cosa testa:** La conversione numerica della porta inserita nel campo di testo.
- **Descrizione:** Inserisce un valore testuale valido nel campo porta e accerta che getServerPort() restituisca il valore convertito in intero primitivo.

- **Test 3 (testGetServerPortInvalid):**

- **Cosa testa:** La reazione a input non numerici nel campo porta.
- **Descrizione:** Imposta caratteri alfabetici nel campo porta. Il test accerta che getServerPort() sollevi NumberFormatException, permettendo al controller di intercettare l'errore di digitazione.

- **Test 4 (testIsDatabaseSource):**

- **Cosa testa:** La discriminazione della sorgente dati selezionata.
- **Descrizione:** Seleziona alternativamente le voci "Da Database" e "Da Archivio" nel combo box della sorgente, verificando che isDatabaseSource() restituisca rispettivamente true e false.

- **Test 5 (testGetServerPortWithWhitespace):**

- **Cosa testa:** La tolleranza agli spazi bianchi accidentali nel campo porta.
- **Descrizione:** Inserisce una stringa con spazi attorno alle cifre (es. " 9090 "). Il test verifica che venga eseguita la sanitizzazione della stringa prima del parsing o che una stringa di soli spazi generi NumberFormatException.

Classe Testata: SummaryPanel

- **Test 1 (testSetPredictionWithValue):**

- **Cosa testa:** La formattazione numerica ad alta precisione del valore predetto.
- **Descrizione:** Passa un valore reale (es. 3.14) a setPrediction(). Il test verifica che l'etichetta mostri il numero formattato con 4 cifre decimali ("3.1400").

- **Test 2 (testSetPredictionWithNull):**

- **Cosa testa:** La gestione dell'assenza di predizione nella visualizzazione.
- **Descrizione:** Invoca `setPrediction(null)`. Il test accerta che l'etichetta mostri il testo segnaposto predefinito "---".
- **Test 3 (testReset):**
 - **Cosa testa:** Il ripristino dei parametri di sessione sul pannello laterale.
 - **Descrizione:** Imposta valori arbitrari e invoca `reset()`. Il test controlla che il contatore dei passi sia azzerato e che la label del valore stimato ritorni a "---".
- **Test 4 (testAddHistoryEntry):**
 - **Cosa testa:** L'aggiornamento della cronologia delle predizioni effettuate.
 - **Descrizione:** Invoca `addHistoryEntry()` fornendo percorso decisionale ed esito numerico. Il test verifica che il `DefaultListModel` sottostante memorizzi un nuovo elemento formattato.

Classe Testata: LogPanel

- **Test 1 (testLogInfoMessage):**
 - **Cosa testa:** La classificazione di messaggi standard di livello informativo.
 - **Descrizione:** Invia un messaggio privo di termini critici e verifica che il documento di log applichi il formato di default INFO.
- **Test 2 (testLogErrorMessage):**
 - **Cosa testa:** L'intercettazione e formattazione dei messaggi di errore.
 - **Descrizione:** Invia una stringa contenente la parola "errore". Il test verifica che il messaggio venga classificato con il livello ERROR.
- **Test 3 (testLogSuccessMessage):**
 - **Cosa testa:** La classificazione dei messaggi di esito positivo.
 - **Descrizione:** Invia una stringa contenente la parola "successo". Il test verifica che il log assegni il livello SUCCESS.
- **Test 4 (testLogWarningMessage):**
 - **Cosa testa:** La classificazione dei messaggi di avviso.
 - **Descrizione:** Invia un testo contenente "warning". Il test accerta che la riga riceva la categorizzazione WARN.
- **Test 5 (testClear):**
 - **Cosa testa:** Lo svuotamento della console di testo.
 - **Descrizione:** Dopo aver scritto diverse righe di log, invoca `clear()` e verifica che la lunghezza del testo nel documento ritorni a 0 caratteri.
- **Test 6 (testLogMessageStylesApplied):**
 - **Cosa testa:** L'applicazione reale degli attributi cromatici nel documento stilizzato.
 - **Descrizione:** Invia un messaggio di errore e uno di successo al `LogPanel`. Il test analizza gli attributi grafici estratti dallo `StyledDocument`, accertando che i caratteri presentino colori di primo piano distinti e conformi alle costanti cromatiche definite.

Classe Testata: TreePanel

- **Test 1 (testDefaultState):**
 - **Cosa testa:** Lo stato iniziale dell'area di rendering grafico.
 - **Descrizione:** Verifica che all'atto della creazione il riferimento `currentTree` sia null, indicando l'assenza di alberi da disegnare.
- **Test 2 (testUpdateTree):**

- **Cosa testa:** L'assegnazione e predisposizione di una nuova struttura da visualizzare.
- **Descrizione:** Passa un'istanza di TreeDTO a updateTree(). Il test verifica che il pannello memorizzi la nuova radice per il calcolo delle coordinate di disegno.
- **Test 3 (testResetTree):**
 - **Cosa testa:** L'azzeramento dell'area di disegno.
 - **Descrizione:** Invoca resetTree() su un pannello che visualizza un albero, accertando che la struttura venga riportata a null.

Package: network

Classe Testata: ServerConnection

- **Test 1 (testInitialNotConnected):**
 - **Cosa testa:** Lo stato iniziale del gestore di rete.
 - **Descrizione:** Verifica che un'istanza appena creata di ServerConnection restituisca false su isConnected() prima di effettuare chiamate di rete.
- **Test 2 (testConnect):**
 - **Cosa testa:** L'instaurazione corretta del canale di comunicazione via socket.
 - **Descrizione:** Avvia un ServerSocket locale e connette la classe client. Il test accerta che il socket si apra e isConnected() diventi true.
- **Test 3 (testClose):**
 - **Cosa testa:** Il rilascio delle risorse e la chiusura degli stream.
 - **Descrizione:** Invoca close() su una connessione attiva. Il test verifica che il socket venga chiuso e che lo stato ritorni a false.
- **Test 4 (testDoubleClose):**
 - **Cosa testa:** L'idempotenza della chiusura della connessione.
 - **Descrizione:** Esegue due volte consecutive il metodo close(). Il test assicura che la seconda chiamata non sollevi eccezioni anomale.
- **Test 5 (testSendWithoutConnection):**
 - **Cosa testa:** Il blocco difensivo sull'invio di dati a canale non attivo.
 - **Descrizione:** Invoca send() senza aver prima aperto il socket. Il test verifica che venga sollevata un'IOException.
- **Test 6 (testReceiveWithoutConnection):**
 - **Cosa testa:** Il blocco difensivo sulla ricezione a connessione chiusa.
 - **Descrizione:** Invoca receive() senza connessione attiva. Il test verifica che il tentativo di lettura fallisca con un'IOException.
- **Test 7 (testSendAndReceive):**
 - **Cosa testa:** Lo scambio effettivo di oggetti serializzati tramite socket.
 - **Descrizione:** Tramite un server mock locale, invia un oggetto stringa e riceve una risposta di eco. Il test verifica l'integrità dell'oggetto ricevuto.
- **Test 8 (testConnectClosesPrevious):**
 - **Cosa testa:** La disconnessione implicita del canale precedente in caso di riconnessione.
 - **Descrizione:** Invoca una seconda volta connect() verso una porta differente. Il test accerta che il socket precedente venga chiuso prima di inizializzare il nuovo.

Classe Testata: PredictionWorker

- **Test 1 (testParseBranchOptionsWithNumbers):**

- **Cosa testa:** Il parsing delle risposte testuali con prefisso numerico.
- **Descrizione:** Passa al parser una stringa con opzioni numerate (es. "0: Sì\n1: No"). Il test verifica che i prefissi numerici vengano rimossi, restituendo pulite le opzioni

"Sì", "No"

- **Test 2 (testParseBranchOptionsWithoutNumbers):**

- **Cosa testa:** Il parsing di opzioni testuali prive di indici numerici.
- **Descrizione:** Passa stringhe su righe separate senza prefisso. Il test accerta che le stringhe vengano conservate senza alterazioni.

- **Test 3 (testParseBranchOptionsWithEmptyLines):**

- **Cosa testa:** La gestione di righe vuote e spazi all'interno del messaggio inviato dal server.
- **Descrizione:** Fornisce una risposta contenente salti riga consecutivi. Il test verifica che le righe prive di testo vengano scartate.

Classe Testata: TableLoaderWorker

- **Test 1 (testTableLoaderWorkerExecution):**

- **Cosa testa:** L'interrogazione asincrona dell'elenco tabelle e il popolamento del modello UI.
- **Descrizione:** Avvia un ServerSocket simulato che risponde al comando 4 inviando una lista di tabelle. Tramite un CountdownLatch, il test sincronizza l'esecuzione del worker in background e verifica che il metodo get() ritorni l'elenco atteso senza deadlock.

Classe Testata: InitTreeWorker

- **Test 1 (testInitTreeWorkerExecution):**

- **Cosa testa:** L'apprendimento o caricamento asincrono dell'albero dal server.
- **Descrizione:** Simula la trasmissione del comando 0 con il relativo nome tabella verso un server mock che risponde "OK". Il test attende il completamento del worker tramite latch e accerta che l'esito restituito dal thread di background coincida con la stringa di conferma.

Package: controller

Classe Testata: MainController

- **Test 1 (testConstructorCreatesConnection):**

- **Cosa testa:** L'orchestrazione iniziale tra vista, listener e connessione di rete.
- **Descrizione:** Istanza MainController passando MainFrame. Il test verifica che l'istanza interna di ServerConnection venga creata e che i componenti della finestra ricevano i rispettivi action listener.

- **Test 2 (testPredictButtonDisabledWithoutConnection):**

- **Cosa testa:** Il vincolo di avvio della predizione in assenza di rete attiva.
- **Descrizione:** Invoca handlePredict() a connessione chiusa. Il test accerta che il controller inibisca l'operazione e mostri una finestra di dialogo di avviso all'utente.

- **Test 3 (testEmptyTableNameShowsWarning):**

- **Cosa testa:** La validazione del nome della tabella prima dell'inizializzazione.

- **Descrizione:** Tenta di avviare handleInitTree() con il campo tabella vuoto. Il test accerta che la richiesta al server venga bloccata con un messaggio di errore.
- **Test 4 (testInvalidPortShowsError):**
 - **Cosa testa:** La gestione coordinata dell'errore di formato porta dalle azioni della GUI.
 - **Descrizione:** Inserisce caratteri non validi nel campo porta e aziona il caricamento tabelle. Il test verifica l'intercettazione dell'eccezione e la segnalazione grafica di errore.
- **Test 5 (testHandleInitTreeSourceArchive):**
 - **Cosa testa:** L'indirizzamento della richiesta verso file serializzato.
 - **Descrizione:** Configura il selettore di origine dati su "Da Archivio" e attiva il caricamento dell'albero. Il test verifica che il controller istanzi il worker con flag di lettura da archivio disattivando le chiamate al database relazionale.